BkenS ERP

Sistema de gestión de agendamiento, personal, servicios y facturación para un centro de estética

Avance 2: modelo de dominio, reglas de negocio y capa de persistencia

Santiago Alonso Perafan
Sistemas
Programación ll
Gabriel Pérez Moreno
Institución Universitaria Antonio José Camacho

Repositorio: https://github.com/Santiagoa1p/Rex
7 de noviembre de 2026

1. Introducción

BkenS ERP es un proyecto académico individual que busca apoyar la gestión básica de un centro de estética y servicios especializados. Este documento presenta el segundo avance: el modelo de dominio orientado a objetos refinado, las reglas de negocio ya implementadas y verificadas con pruebas unitarias, la incorporación de control de acceso por roles y la definición de la capa de persistencia mediante interfaces DAO.

El documento sigue la estructura solicitada: planteamiento del problema, justificación, objetivos, metodología, el diagrama del modelo y las referencias. Además, incluye una sección que relaciona cada observación recibida sobre la primera entrega con el cambio realizado y una sección que explica por qué el sistema se denomina ERP aunque su núcleo actual sea el agendamiento de turnos.

2. Planteamiento del problema

En los centros de estética y servicios especializados es frecuente que las citas, la disponibilidad del personal y los pagos se gestionen con medios manuales o dispersos, como agendas en papel, mensajes de chat u hojas de cálculo independientes. Esta forma de trabajo dificulta saber con certeza qué especialista está libre en un horario dado, favorece los cruces y las citas sin confirmar, y deja los cobros desconectados del servicio efectivamente prestado, lo que complica el seguimiento de los ingresos. Además, sin un control de acceso, cualquier persona que use el registro puede consultar o modificar la información.

La primera versión del proyecto representó estas entidades (clientes, especialistas, servicios, turnos y transacciones), pero de forma puramente descriptiva: el estado del turno y el método de pago eran textos libres, el horario del especialista era un texto que el programa no podía interpretar, y no existían reglas que impidieran operaciones inválidas, como cobrar un turno cancelado.

De lo anterior se desprende la pregunta que orienta este avance: **¿cómo modelar, con orientación a objetos, el agendamiento y la facturación de un centro de estética de manera que las reglas (disponibilidad, ciclo de vida del turno y cobro) se verifiquen automáticamente y que el medio de almacenamiento pueda cambiar sin reescribir el dominio?**

3. Justificación

3.1 Justificación del proyecto

3.2 ¿Por qué se denomina ERP?

Un sistema ERP (*Enterprise Resource Planning*) integra, sobre un conjunto de datos compartido, los procesos de distintas áreas de una organización, de modo que lo registrado en un proceso queda disponible para los demás (Monk y Wagner, 2013). BkenS ERP adopta ese principio de integración: los procesos que atiende no son aplicaciones aisladas, sino que comparten las mismas entidades.

- **Agenda:** el turno enlaza al cliente, al especialista, al servicio y, cuando se cobra, a su transacción.
- **Personal:** la disponibilidad pertenece al especialista y se consulta con *estaDisponible()*.
- **Catálogo de servicios y facturación:** la transacción toma su monto del precio del servicio, sin que nadie lo digite a mano.
- **Control de acceso:** *Usuario* y *Rol* determinan quién opera el sistema.

Se reconoce expresamente que, en su estado actual, el núcleo del sistema es el agendamiento de turnos y que se trata de un ERP de alcance reducido, pensado para una pequeña organización. Módulos propios de un ERP más amplio, como inventario de insumos, reportes financieros o nómina, no están implementados y se consideran trabajo futuro. El nombre describe, por tanto, la arquitectura integradora y modular del proyecto, no la amplitud funcional que tiene hoy.

4. Objetivos

4.1 Objetivo general

Diseñar e implementar en Java un modelo de dominio orientado a objetos, con sus reglas de negocio básicas, para la gestión de agenda, personal, servicios, facturación y acceso de un centro de estética, dejándolo preparado para adoptar la arquitectura MVC y una base de datos.

4.2 Objetivos específicos

- Modelar las entidades del dominio y sus relaciones en un diagrama de clases UML, usando herencia, composición, enumeraciones y multiplicidades.
- Implementar las reglas de disponibilidad del especialista, del ciclo de vida del turno (confirmar, cancelar, cobrar) y del cobro basado en el precio del servicio.
- Incorporar autenticación con roles, almacenando las contraseñas como valores hash.
- Definir la capa de persistencia mediante interfaces DAO independientes del medio de almacenamiento.
- Verificar las reglas con pruebas unitarias automatizadas (JUnit 5).

- Gestionar el desarrollo con Git de forma incremental, de modo que el historial de commits refleje la evolución del proyecto.

5. Metodología

Se siguió un enfoque incremental e iterativo (Sommerville, 2016): el sistema se construye en incrementos pequeños, cada uno con un propósito único. En cada ciclo se define el cambio en el diseño, se implementa, se prueba y se registra en el control de versiones. La primera entrega estableció la línea base (modelo inicial y menú de consulta); este avance refina el modelo y agrega reglas, seguridad y persistencia abstracta; la tercera entrega implementará MVC y la base de datos.

5.1 Herramientas

Aspecto	Herramienta
Lenguaje	Java 17
Construcción y dependencias	Apache Maven
Entorno de desarrollo	Visual Studio Code
Pruebas unitarias	JUnit 5 (Jupiter) ejecutado con Maven Surefire (JUnit Team, s. f.)
Control de versiones	Git y GitHub: repositorio local y remoto (Chacon y Straub, 2014)
Modelado	UML (Object Management Group [OMG], 2017); el diagrama se mantiene en el repositorio en notación Mermaid y las figuras de este documento lo representan en UML

5.2 Criterios de trabajo

- **Commits por cambio lógico:** cada commit agrupa un cambio con un solo propósito que compila y funciona por sí solo, ni línea por línea ni todo de una vez. Los mensajes usan prefijos convencionales (*feat*, *refactor*, *build*, *chore*).
- **Rama de trabajo:** el desarrollo se realiza en la rama *Propuesta*; la integración a *main* se hace mediante Pull Request una vez verificada la entrega.
- **Pruebas donde hay reglas:** se prueba la lógica con decisiones (disponibilidad, transiciones de estado, cobro, autenticación). No se prueban getters, setters ni enumeraciones sin lógica.
- **Verificación antes de cada commit:** compilación con *mvn compile* y pruebas con *mvn test*.
- **Fuente única del diseño:** el código y el diagrama se actualizan en paralelo para que describan el mismo modelo.

6. Atención a la retroalimentación de la primera entrega

La siguiente tabla relaciona cada observación del docente con el cambio realizado.

Observación	Cambio realizado
Presentar el documento del avance en PDF con planteamiento, justificación, objetivos, metodología, diagrama y referencias APA.	Este documento.
Agregar constructores y métodos al diagrama (*confirmar()*, *cancelar()*, *cobrar()*, *estaDisponible()*).	La Figura 1 muestra constructores y métodos. Los cuatro métodos están implementados y probados.
Convertir *estado* y *metodoPago* en enums.	Se crearon *EstadoTurno* y *MetodoPago* y reemplazaron a los *String*.
Poner en *Turno* los atributos de sus asociaciones.	*Turno* declara *cliente*, *especialista*, *servicio* y *transaccion* (esta última asignada solo por *cobrar()*).
Modelar el horario del especialista como clase.	Se creó *HorarioDisponibilidad* (día, hora de inicio y fin), relacionada por composición con *Especialista*.
Agregar Usuario y Rol.	Se crearon *Usuario* (con autenticación y contraseña en hash) y el enum *Rol*.
Separar la persistencia en DAOs.	Se definieron la interfaz genérica *Dao<T>* y una interfaz por entidad (Figura 2). Las implementaciones quedan para el siguiente incremento.
Explicar por qué es un “ERP”.	Sección 3.2.

7. Diseño del sistema

7.1 Modelo de dominio

La Figura 1 presenta el modelo de dominio. *Persona* es una clase abstracta que concentra los datos comunes; *Cliente* y *Especialista* la especializan. El especialista compone una lista de *HorarioDisponibilidad*, que deja de ser texto y pasa a ser información que el programa puede evaluar. El *Turno* es la entidad central: referencia al cliente, al especialista y al servicio, tiene un *EstadoTurno* y, cuando se cobra, una única *Transaccion* (relación 0..1) que registra el *MetodoPago*.

7.2 Reglas de negocio implementadas

El ciclo de vida del turno y la disponibilidad se encapsularon en métodos de las propias entidades, de modo que ninguna otra parte del programa pueda dejar un turno en un estado inconsistente. Las

transiciones inválidas lanzan una excepción, porque representan un error de lógica de quien llama y no una situación normal que pueda ignorarse.

Operación	Condición	Resultado	Si no se cumple
Turno.confirmar()	Estado PENDIENTE	Estado CONFIRMADO	Lanza *IllegalStateException*
Turno.cancelar()	Estado PENDIENTE o CONFIRMADO	Estado CANCELADO	Lanza *IllegalStateException*
Turno.cobrar(...)	Estado CONFIRMADO	Crea la *Transaccion* con el precio del servicio y pasa a COMPLETADO	Lanza *IllegalStateException* (también impide cobrar dos veces)
Especialista.estaDisponible(fecha, hora, duracion)	Algún horario coincide con el día de la semana y contiene el servicio completo (inicio y fin dentro de la franja)	*true*	Devuelve *false* (día sin horario, fuera de franja, duración inválida o servicio que cruza la medianoche)

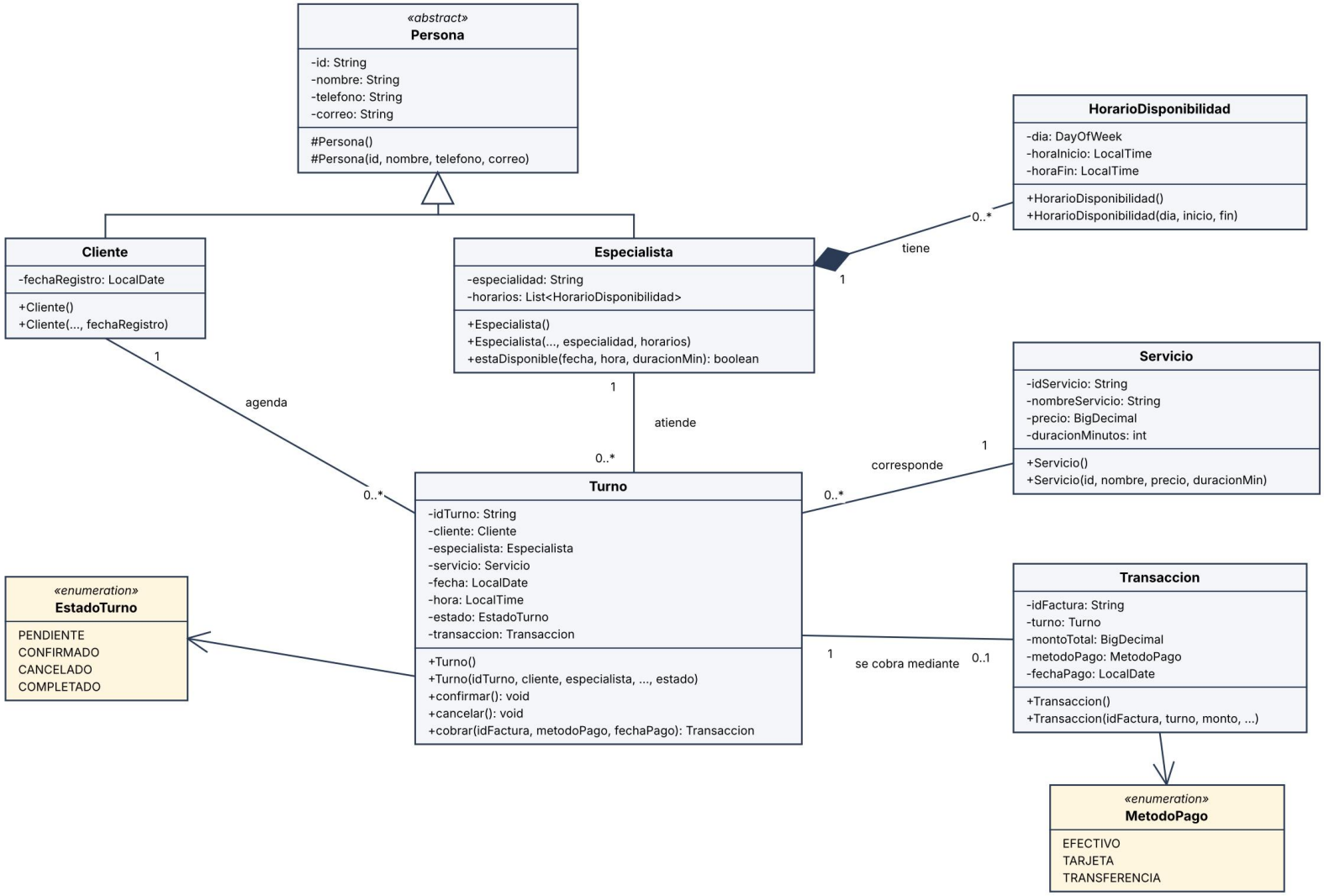
Un servicio que termina exactamente a la hora de cierre es válido (por ejemplo, 16:00 a 17:00 en un horario hasta las 17:00), porque la hora de cierre es el instante en que termina la atención. Por ahora *estaDisponible()* solo compara contra los horarios del especialista; la detección de choques con otros turnos queda para cuando exista una agenda persistida de la cual consultar.

7.3 Acceso y persistencia

Usuario guarda el nombre de usuario, el rol y un hash SHA-256 de la contraseña, nunca el texto plano; *autenticar()* verifica las credenciales y que la cuenta esté activa. Los roles son administrador, recepcionista y especialista. La persistencia se separa en la interfaz genérica *Dao<T>* (guardar, buscar por id, listar y eliminar) y una interfaz por entidad; *TurnoDao* añade la consulta por fecha y *UsuarioDao* la búsqueda por nombre de usuario. Las entidades implementan *Serializable* con *serialVersionUID*, de modo que la primera implementación puede guardar archivos locales y la siguiente, una base de datos. La Figura 2 resume esta estructura.

Figura 1

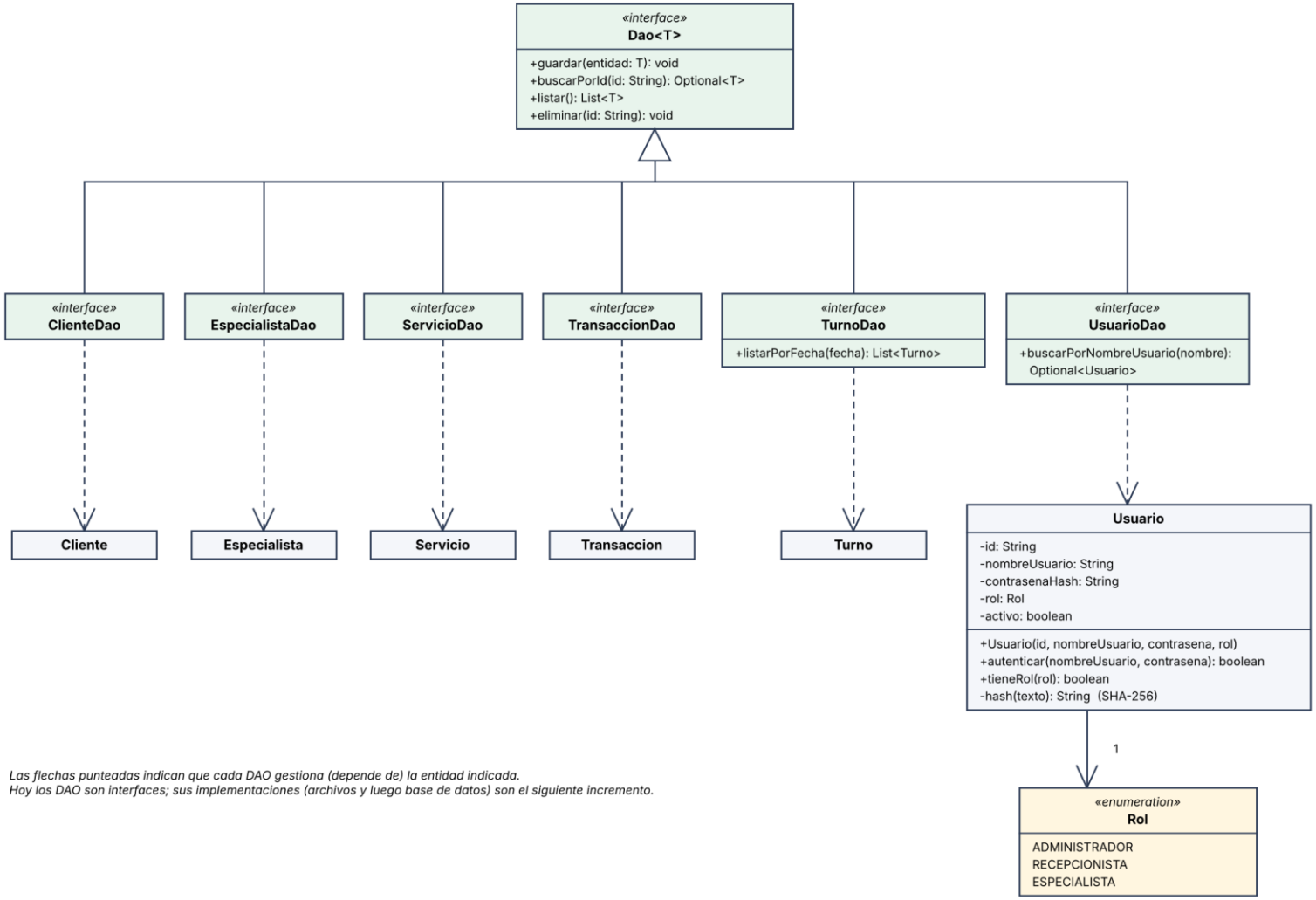
Modelo de dominio de BkenS ERP



Nota. Elaboración propia en notación UML. Los constructores de las subclases se abrevian con "…": reciben primero los datos de *Persona* (id, nombre, teléfono, correo). Las flechas abiertas indican que la clase usa la enumeración.

Figura 2

Control de acceso y capa de persistencia (DAO)



Nota. Elaboración propia en notación UML. Las implementaciones concretas de los DAO son el siguiente incremento.

8. Verificación y estado del proyecto

8.1 Pruebas unitarias

Las reglas se verifican con 25 pruebas automatizadas con JUnit 5, todas aprobadas al momento de este documento. Cada prueba cubre un caso límite distinto de la regla.

Clase probada	Pruebas	Casos cubiertos
Especialista	7	Dentro del horario, cierre exacto, antes de abrir, después del cierre, día sin horario, lista de horarios vacía, cruce de medianoche.
Turno	12	Confirmar (1 válido y 3 inválidos), cancelar (2 válidos y 2 inválidos), cobrar (resultado completo, pendiente, cancelado y doble cobro).
Usuario	6	Credenciales correctas, contraseña o usuario incorrectos, cuenta inactiva, contraseña no almacenada en texto plano, consulta de rol.

8.2 Estado actual

Implementado	Pendiente
Modelo de dominio con enums, composición y herencia. Reglas de disponibilidad, ciclo de vida del turno y cobro. *Usuario* y *Rol* con autenticación. Interfaces DAO para las seis entidades. Menú de consola de consulta (agenda, servicios, caja) con datos de ejemplo en memoria.	Implementaciones de los DAO (archivos y luego base de datos). Integrar *Usuario* y los DAO al menú (inicio de sesión y registro de turnos). Detección de choques entre turnos. Arquitectura MVC y base de datos (tercera entrega). Módulos adicionales de un ERP (inventario, reportes).

8.3 Limitaciones

- El hash SHA-256 sin sal es suficiente para un proyecto académico, pero no para producción; allí se recomiendan funciones de derivación adaptativas como bcrypt o Argon2 (OWASP Foundation, s. f.).
- Los datos del menú viven en memoria y se pierden al cerrar el programa hasta que se implementen los DAO.
- El sistema no tiene interfaz gráfica; el menú de consola es provisional.

8.4 Próximos pasos

La tercera entrega implementará la arquitectura MVC (Fowler, 2002), separando el modelo actual de las vistas y los controladores, y conectará una base de datos relacional escribiendo nuevas implementaciones de las interfaces DAO ya definidas, sin modificar las entidades.

9. Referencias

Alur, D., Crupi, J., y Malks, D. (2003). *Core J2EE patterns: Best practices and design strategies* (2.ª ed.). Prentice Hall.

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/en/v2

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

JUnit Team. (s. f.). *JUnit 5 user guide*. https://junit.org/junit5/docs/current/user-guide/

Monk, E., y Wagner, B. (2013). *Concepts in enterprise resource planning* (4.ª ed.). Cengage Learning.

Object Management Group. (2017). *Unified Modeling Language* (Versión 2.5.1). https://www.omg.org/spec/UML/2.5.1

Oracle. (s. f.). *Package java.time*. En *Java Platform, Standard Edition 17 API specification*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/time/package-summary.html

OWASP Foundation. (s. f.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.